Diabetic Retinopathy Classification using Deeply Supervised ResNet

Debiao Zhang
School of Computer Science and Technology
Harbin Institute of Technology
Harbin, China
zhangdebiao.hit@gmail.com

Wei Bu
Department of Media Technology and Art
Harbin Institute of Technology
Harbin, China
buwei@hit.edu.cn

Xiangqian Wu
School of Computer Science and Technology
Harbin Institute of Technology
Harbin, China
xqwu@hit.edu.cn

***Abstract*—Diabetic retinopathy (DR) is the leading cause of blindness in the working-age population all over the world. Classifying DR is a time-consuming work, which requires an experienced ophthalmologist to evaluate color fundus photographs of retina. In this paper, we propose a deeply supervised ResNet approach to classify the severity of DR automatically. In this new convolutional neural networks (CNN) architecture, we add three sets of additional side-output layers to intermediate hidden layers of a 11-layer ResNet. By introducing these deeply supervised layers, we can provide additional regularization during training network. More importantly, we can perform multi-scale learning by leveraging the predictions of intermediate supervised layers, thus improving the final performance. Furthermore, to combat the issue of class-imbalance in the dataset, we adopt cost sensitive learning and an oversampling method of cropping images. We train and evaluate our network on the publicly available Kaggle dataset, and the results show that our method outperforms the state-of-the-art method.**

Keywords*—*Diabetic Retinopathy Classification; Deeply Supervised Network; ResNet; Cost Sensitive Learning; Multi-Scale Learning

I. INTRODUCTION

Diabetic retinopathy is the most frequently occurring complication of diabetes mellitus and a leading cause of blindness [2]. Uncontrolled diabetes and its complication finally lead to DR [3]. The clinical diagnosis of DR needs an ophthalmologist to examine carefully the color fundus images of patients and then grade the severity. The accuracy and timing of the diagnosis is very important to the treatment. If detected accurately and early enough, the progress to visual impairment can be delayed effectively through some healthcare measures.

According to the newest UK national criterion, DR can be classified into five stages with increasing severity: normal, mild, moderate, severe and proliferative (For simplicity, we also use 0 to 4 to indicate these classes, respectively). Fig. 1 demonstrates color fundus image examples of different stages of DR. Classification of DR involves the weighting of visual features of several lesions presenting in color fundus image of patients [1]. These lesions include microaneurysms, hemorrhages, hard exudates, soft exudates, neovascularization, and so on. Fig. 2 shows these lesions. From Fig. 2 we can see that some lesions

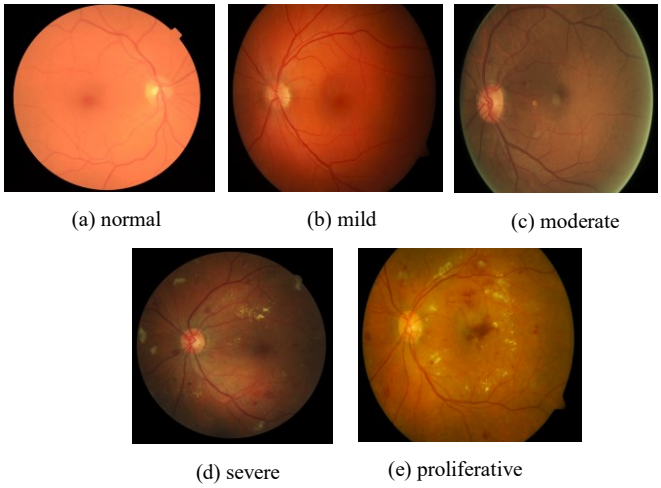


Fig. 1. Examples of different stages of DR.

are quite subtle, especially for microaneurysms, which makes clinical diagnosis difficult and time consuming for an ophthalmologist. However, computers can evaluate and classify DR in real time if a classification model is well trained.

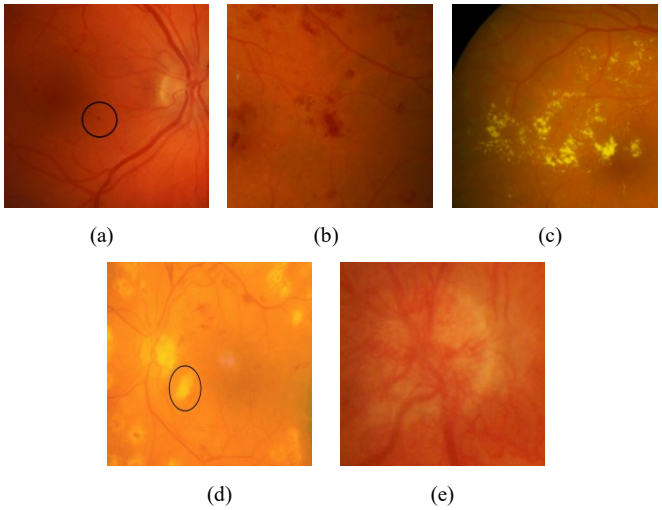


Fig. 2. Typical lesions in color fundus image with DR. (a) microaneurysms; (b) hemorrhages; (c) hard exudates; (d) soft exudates; (e) neovascularisation.

978-1-5386-0435-9/17/$31.00 ©2017 IEEE

Authorized licensed use limited to: University of Liverpool. Downloaded on November 16,2022 at 18:13:22 UTC from IEEE Xplore. Restrictions apply.

In this paper, we propose a new CNN network architecture, inspired by the work from deeply supervised nets (DSN) [4] and ResNet [5], to classify DR from end to end. We conduct the experiment on the large public dataset provided from Kaggle. Despite our network is shallower than that used in Pratt et al [1], the results exhibit that our network outperforms significantly than their method on this benchmark dataset.

The remainder of this article is organized as follows: Sect. II discusses the related works on this field that is followed by a detailed description of the proposed methodology in Sect. III. In Sect. IV, experimental results are presented. Finally, conclusion and future work are given in Sect. V.

II. Related Work

In this section, we first discuss different DR classification methods. Because the dataset used in the experiment is highly class-imbalanced, which has a great impact on the performance of the trained model, we also explore different methods to address this problem. At last, we present the two networks that directly motivate our network.

A. Previous DR Classification Methods

Research on automatic classification of DR has been an active area in medical image processing during the last decades [3]. Many researchers have proposed different methods to classify DR. These methods can be mainly classified into three categories according to different classification criterions: the first is a binary classification, i.e., with or without DR; the second is a three-class classification of normal, non-proliferative or proliferative DR; the newest and most widely used criterion is the five-class classification described in Sect. I.

For the binary classification, Gardner et al [6] used pixel intensity as input feature of a Neural Network, and got sensitivity and specificity results of 88.4% and 83.5% respectively on a small dataset of around 200 images. Roychowdhury et al [7] proposed a two-step hierarchical classification approach that combines four machine learning algorithms of Gaussian Mixture model (GMM), k-nearest neighbor (kNN), support vector machine (SVM) and AdaBoost, and achieved a result of 100% sensitivity, 53.16% specificity, and 0.904 AUC. Priya et al [8] classified fundus images with DR into proliferative or non-proliferative DR, they first extracted features of blood vessels, hemorrhages and exudates, then fed these features in to three classification models of Probabilistic Neural Network (PNN), Bayesian Classifier and Support vector machine (SVM), achieving an accuracy of 89.6%, 94.4% and 97.6%, respectively.

As for the three-class classification, Nayak et al [9] extracted features such as the area of exudates and the area of blood vessels together with textures, and fed them in to a Neural Network. On a dataset of 140 images, they achieved an accuracy of 93%, sensitivity of 90% and specificity of 100%.

Most of recent methods concentrate on five-class classify-cation. Acharya et al [10] extracted features using a higher-order spectra method and classified images using SVM. This method reported a sensitivity of 82%, specificity of 88% and accuracy of 82%. Adarsh et al [11] used the area of lesions and texture features and trained multiclass SVMs for classification. As deep learning emerged in recent years, Pratt et al [1] proposed to classify DR using a 13-layer CNN and evaluated the performance of their network on the large Kaggle dataset. We will compare our method with this method in Sect. IV.

B. Class-Imbalance Feature Learning

Imbalanced datasets exist in many real-world domains, such as spotting unreliable telecommunication customers, text classification, information retrieval, medical image processing, and so on [12]. Most of classification algorithms use objective functions based on regularized and continuous 0-1 loss function. Without handling the imbalance problem, conventional methods tend to be biased to the majority classes with poor accuracy for the minority classes [13].

Many solutions to the class-imbalance issue have been proposed both at data and algorithm levels [12]. At data level, the solutions include different forms of resampling such as oversampling, undersampling and combinations of them. At algorithm level, cost-sensitive learning is often used, in which different weights are assigned to different classes when calculating the value of loss function. In [15, 16], the authors combine different cost-sensitive learning methods with deep learning to perform contour detection. In [21], the authors also use a cost-sensitive loss function to train a CNN network when performing saliency detection.

C. Deeply Supervised Nets and ResNet

Deep learning has become increasingly popular in both academic and industrial areas during the past years. Various domains including pattern recognition, computer vision, and natural language processing have witnessed the great power of deep neural networks.

DSN [4] and ResNet [5] motivate the network used in this paper. In [4], the authors introduce “companion” objective functions at each hidden layer to combat the following three issues existing in traditional CNN architectures: transparency in the effect intermediate layers have on overall classification; discriminativeness and robust-ness of learned features, especially in early layers; training effectiveness in the face of “vanishing” gradients. The idea of DSN has been successfully applied to many computer vision tasks, such as scene text detection [18], saliency detection [19, 27], and edge detection [20]. In [5], the authors present a residual learning framework to ease the training of extremely deep networks. In this network, they explicitly reformulate the layers as learning residual functions with reference to the inputs, instead of learning unreferenced functions. Many methods [22, 23, 24, 25] based on ResNet are proposed at different situations.

III. Proposed Method

In this section, we describe our proposed DR classification method in detail.

A. Network Architecture

As demonstrated in [4], inserting additional classification layers after each hidden layer can improve the performance of

Authorized licensed use limited to: University of Liverpool. Downloaded on November 16,2022 at 18:13:22 UTC from IEEE Xplore. Restrictions apply.

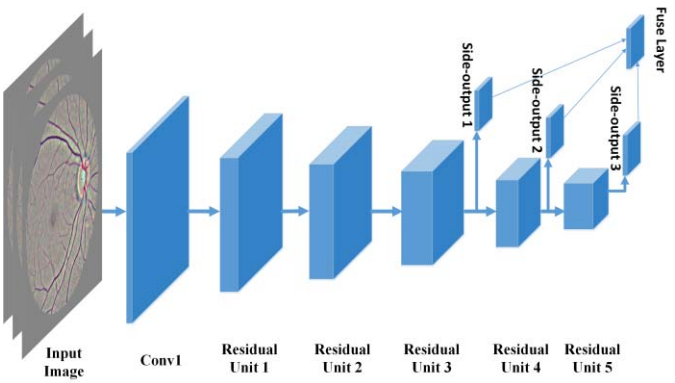


Fig. 3. Framework of our network for DR classification. Three sets of side-output layers and a fusion layer are inserted after last three residual units. When training the network using error back-propagation, the errors produced by different classifiers are accumulated at the aggregation points.

CNN in some degree. Besides, ResNet has been seen to achieve state-of-the-art performance in ImageNet challenge while keeping a simpler computation complexity than many other network architectures. Based on these good properties, we develop our convolutional network architecture, which is illustrated in Fig. 3. Our network consists of five pre-activation residual units [14]. In each of these units exist two convolutional layers. Fig. 4 shows the units in detail. Three additional sets of side-output layers are inserted after the third, fourth, and fifth residual units, respectively. We do not add side-output layers after the first two residual units, because the size of feature maps produced by these units is quite large, more importantly, the layers in these units mainly learns low-level visual features that are not comprehensive enough for classification. Furthermore, we add a fusion layer to combine outputs from the multi-scale side-output layers. Table. I shows the detailed architecture except for the additional side-output layers and fusion layer.

The exact structures of the side-output layers are as follows: all the side-output layers consist of an average pooling layer, a fully connected layer with five units and a softmax layer with cross entropy loss; the filter size and stride of the three average pooling layers are 7×7, 3, 5×5, 3, and 7×7, 1, respectively. Besides, these side-output layers are inserted after BatchNorm and ReLU layers. The average sum of outputs of the three fully connected layers is fed in to the softmax layer of the fusion layer.

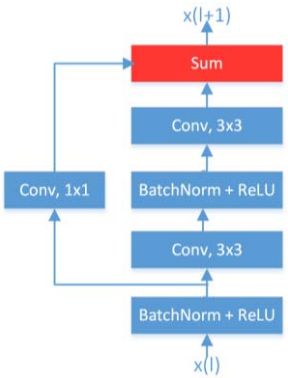


Fig. 4. Illumination of residual units used in our network. X(i) indicates inputs of residual unit i. The number of filters in the shortcut are the same as ResNet. Note that unit 1 does not consist 1×1 convolutional layer in the shortcut.

TABLE I. NETWORK ARCHITECTURE EXCLUDING SIDE-OUTPUT LAYERS AND FUSION LAYER. CONV2 TO CONV6 INDICATE THE CORRESPONDING RESIDUAL UNITS. THE FIRST CONVOLUTIONAL LAYER OF THE LAST FOUR RESIDUAL UNITS HAS A STRIDE OF 2.

layer name	*filter size/#filter*	*depth*	*output size*
conv1	7×7/64 (stride 2)	1	224×224
max pool	3×3/- (stride 2)	0	112×112
conv2_x	3×3/64	2	112×112
conv3_x	3×3/128	2	56×56
conv4_x	3×3/256	2	28×28
conv5_x	3×3/256	2	14×14
conv6_x	3×3/512	2	7×7

B. Formulation

Our goal is to train a network that learns features from which it is possible to classify color fundus images into different stages of DR. Let us suppose the network has S side-output layers, and denote W and $w = (w^0, w^1, \ldots, w^S)$ as the parameters of standard network layers and the S side-output layers, respectively. Each side-output layer has a classifier, and produces a cross-entropy loss ℓ^i_{side}.

Considering that the dataset used in the experiment is highly class-imbalanced, we adopt cost-sensitive learning when training the network. For simplicity, we introduce a constant class-balancing weight β^i to class i to offset the imbalance between different classes. Then we get the following multi-class balanced cross-entropy loss function as (1):

$$\ell^i_{side}(W, w^i) = -\frac{1}{N}\beta^c \log(\hat{p}^c_n | X_n, c; W, w^i) \qquad (1)$$

where N indicates the number of training samples, X_n and c indicate input image and its class label, respectively. β^c denotes the weight of class c. $\hat{p}^c_n$ indicates the probability of image sample X_n being classified as class c. The detailed setting of class-balancing weight β will be introduced in Sect. IV. ℓ_{fuse}, the loss of fusion layer, can be also calculated as (1). Taking everything into account, the overall loss of the network can be defined as follows:

$$\ell(W, w) = \sum_{i=1}^{S} \alpha_i \ell^i_{side}(W, w^i) + \alpha_{fuse}\ell_{fuse}(W, w) \quad (2)$$

where α_{fuse} and α_i are loss weight of fuse layer and side-output layer i. We set all the loss weight to one in our experiment.

When optimizing the objective function of (2) via stochastic gradient descent, we need to modify the back-propagation equation of the new cross-entropy loss layer. Quite simply, all the work is to add the class-balancing weight β to the standard back-propagation equation.

At testing phase, given an input image X, we classify the image according to the prediction of fusion layer.

IV. EXPERIMENTS

In this section, we evaluate our network on the dataset from Kaggle and compare with the newest method performed on this dataset from [1]. First, we introduce two measures used in our experiment to combat class-imbalance of the dataset. Then, we describe the detail implementation of our method. At last, we show the results and compare with [1].

Due to lack of labels of the official test set, we only use the official training set, which contains more than 35000 images.

Authorized licensed use limited to: University of Liverpool. Downloaded on November 16,2022 at 18:13:22 UTC from IEEE Xplore. Restrictions apply.

Then we split it into our own training, validation and test sets using the method of stratified sampling. The three subsets each consist of 60%, 20% and 20% images of the official training set, respectively.

A. Addressing Class-Imbalance Issue

As mentioned at Sect. III, the dataset used in the experiment is highly class-imbalanced. Table. II shows the original skewed distribution between classes of the split training set. Without handling this issue, the model will tend to classify all samples into the majority classes.

We first perform before-mentioned cost-sensitive learning by add class weight β to loss function, where $\beta = (\beta_1, \beta_2, \ldots, \beta_{|C|})$. $|C|$ denotes the number of classes, and $\beta_i = 1 - p_i$ denotes the weight for class i, where p_i is the percentage of class i in the training set.

In addition to cost-sensitive learning, we also over-sample the minority classes (i.e., class 1, 2, 3, 4) to get a more balanced distribution. As seen in the "original" column of Table. II, the class 0 consist of over 73% images of the original dataset. When performing oversampling, we first resize the "preprocessed" images of minority classes to 600×600. Similar to [17], we then crop out 512×512 patches at the center and 4 corners of every image of minority classes and its horizontal reflection. After cropping, every image of minority classes produces 10 patches. Then we randomly choose N patches, where N indicates the "Sample Factor" in Table. II. After oversampling, we get a relatively balanced dataset. Note that we do not perform oversampling on validation and test sets, so their distributions are the same as "original" column in Table. II shows.

B. Implementation

We implement our network using CAFFE [26] library and build on top of the implementations of ResNet [5] and DSN [4]. Considering that our network is shallow and color fundus images are quite different from the natural images such as that from ImageNet and CIFAR-10, we train the network from scratch.

1) Data Preprocessing

The images in this dataset are taken under a variety of imaging conditions from patients of varying ethnicity and age groups. Based on the properties of the dataset, we preprocessed all the images as follows:

a) Cropping Region of Interest (ROI): Because the raw image has large black borders (as shown in Fig. 5 (a)), which are unneccessary for classification, we crop the center ROI for each image.

b) Resizing: Most of the images in the dataset have a resolution of 3000×3000 or so, so we resize the cropped image to 512×512, which is a trade-off between the process ability of GPUs and the intricate features in color fundus images. If the image size is too small, the features of some tiny discriminative regions such as microaneurysms will be hard to extract.

c) Color and Brightness Normalization: The brightness and color of some images present a wide variation, which is irrelevant to DR classification, so we perform color and brightness normalization to all the images. From Fig. 5 (b) we can see that this measure normalizes images with different color and brightness, thus reducing these noises in a degree when training the network.

2) Data Augmentation

To overcome overfitting, we adopt the following data augmentation measures to the training data set during training:

a) Flipping: Random flip an image horizontally, or vertically or both or don't flip.

b) Smoothing: Random smooth an image with a 3×3 Gaussian filter.

c) Contrast Adjustmemt: Random adjust the contrast.

d) Rotation: Random rotate an image by a degree between [0, 360].

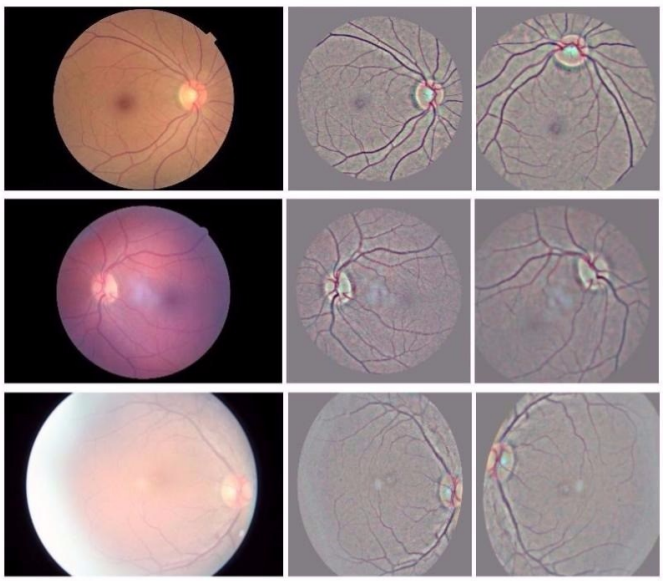
e) Cropping: Crop the 512×512 image into size of 448×448 with random offsets.

Fig. 5 shows some examples of preprocessed and augmented images.

3) Network parameters

TABLE II. DISTRIBUTION BETWEEN CLASSES BEFORE AND AFTER OVERSAMPLING

class	Original		After Oversampling		
	Sample Number	*Percentage (%)*	*Sample Number*	*Sample Factor*	*Percentage (%)*
0	15537	73.55	15537	-	32.2
1	1465	6.94	7325	5	15.2
2	3175	15.03	15875	5	32.9
3	523	2.48	5230	10	11.0
4	424	2.01	4240	10	8.8



(a) (b) (c)

Fig. 5. Examples of preprocessed and augmented images. (a) is the source image, (b) is the preprossed image, (c) is the augmented image.

Authorized licensed use limited to: University of Liverpool. Downloaded on November 16,2022 at 18:13:22 UTC from IEEE Xplore. Restrictions apply.

When tuning the deep model hyper-parameters, we mainly follow the advice from [28] and reference to the settings in original ResNet [5] and DSN [4]. After a rough search with a small validation set which is sampled from training set, we fix the hyper-parameters as follows: base learning rate (0.2), mini-batch size (64), momentum (0.9), weight decay (0.0001), and maximum training iterations (25000). We divide learning rate by 2 every 5000 iterations. Besides, we also explore different values of the class weights in cost-sensitive learning, sample factors in oversampling, the loss weights of prediction layers, and finally fix the value of them described in the corresponding section. Furthermore, we also try to add additional convolutional layers to the side-output layers, but this do not improve the performance of the network while increasing the complexity of the network, so we retain the structure of these side-outputs as described in Sect. III.

4) Training and Testing

We train our network on the training dataset via stochastic gradient descent with Nesterov acceleration and monitor the performance of the network on validation dataset during training. As the dataset is class-imbalanced (even though oversampling is performed) and cost-sensitive learning is used, the loss and accuracy on validation dataset may wiggle in a big range at the first several epochs, but the network still converges if everything is set correctly. On a single NVIDIA Tesla K40c GPU, the whole training takes about 2.5 days, which is much shorter than [1] (their model takes more than 15 days to train as described in their paper).

When testing an image, we only crop the image at center for fairness when comparing with [1].

C. Results

After the network converges, we evaluate the performance of our network on the test dataset, which contains about 7000 images. We adopt the evaluation metrics introduced in [1], i.e., accuracy (ACC), sensitivity (SEN), and specificity (SPE) for this five class classification problem. Specially, they define sensitivity as the number of images correctly identified as having DR out of the true amount with DR, and specificity as the number of images correctly identified as not having DR out of the true amount without DR. In addition to the overall accuracy, we also introduce the "average precision (AP)" of different classes:

$$\overline{A} = \frac{1}{|C|}\sum_{i=1}^{|C|} a_i \tag{3}$$

where α_i indicates the accuracy of class *i*. Considering the class-imbalance of the dataset, the average accuracy can evaluate the performance of models better.

In order to verify the role of deep supervision and the effectiveness of cost-sensitive learning together with oversampling, we also compare our deeply supervised network with another two ResNets (i.e., ResNet_1 and ResNet_2 in Table. III) that both have the same depth. Furthermore, we present the classification results of each side-output layer and the fusion layer as well. Table. III shows the comparison results between different layers, the two ResNets and the method of [1]. In Fig.6 we show the detailed classification results of each class in the form of confusion matrix. Note that the results of [1] listed below are all from their paper.

TABLE III. CLASSIFICATION RESULTS OF SIDE-OUTPUT LAYERS, FUISON LAYER, TWO RESNETS, AND [1]. IN RESNET_1, WE DO NOT PERFORM COST-SENSITIVE LEARNING, OVERSAMPLING, OR DEEP SUPERVISION. IN RESNET_2, WE ONLY EXCLUDE DEEP SUPERVISION FROM OUR METHOD.

Method/Layer	ACC (%)	SPE (%)	SEN (%)	AP (%)
Pratt el al. [1]	75	95	30	34.1[a]
ResNet_1	80.7	**99.3**	43.9	38.7
ResNet_2	73.2	85.0	**73.0**	52.0
Side-output 1	**81.0**	94.5	65.1	52.0
Side-output 2	80.4	93.6	66.7	54.1
Side-output 3	80.0	93.1	67.9	**55.0**
Fusion Layer	80.8	94.0	66.5	54.0

[a.] AP of [1] is computed by us according to the confusion matrix, seen in Fig. 6 (b)

From Table. III, we can see that our model performs much better than [1] in terms of ACC, SEN and AP, especially that SEN and AP are about two times of [1]. The SPE of [1] is slightly higher than our method, because their model tend to classify the images into the majority class 0, which can be seen in Fig. 6 (b). Although ResNet_1 has a relatively higher ACC and SPE than ResNet_2, the improvement of SEN and AP made by ResNet_2 is much more significant over ResNet_1. The reason is that ResNet_2, which combines with cost-sensitivity learning and oversampling, has a more accurate classification of the minority classes. Compared with ResNet_2, our model performs better in terms of ACC, SPE and AP. We attribute the performance improvement to the supervised side-output layers that learn the classification features at certain scale. However, the poor classification performance in class 1 result in the lower SEN. As for the results of different prediction layers, the layers at lower level have higher ACC and SPE, whereas the higher level layers have higher SEN and AP. After investigating the confusion matrices of these side-output predictions, we find that the lower level side-output layers have a better classification of class 0, 1, and 2, while the higher level layers classify the samples of class 3 and 4 more accurately. The fusion layer offers a balance between these side-output predictions. Besides, the predictions of different side-output layers are quite similar, this is because the network used in our experiments is relatively shallow, the difference between the features learned by side-output layers is small. As the network goes deeper, the role of deep supervision should be more outstanding.

From Fig. 6, we can clearly see that our model has a higher

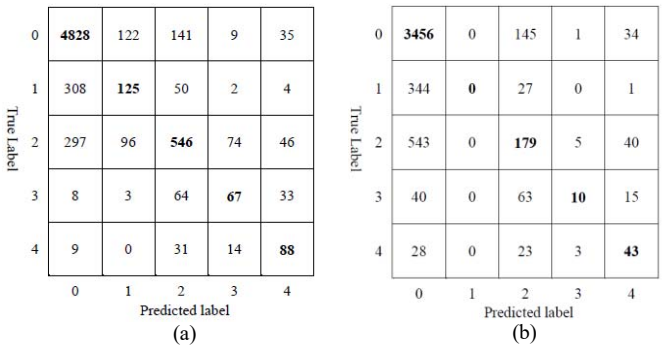


Fig. 6. Confusion matrix of classification results. (a) is the preditions of our method, (b) is the result of [1]. Our model has a significantly higher classification accuracy of minority classes. Note that none of images of class 1 is correctly classified in [1].

Authorized licensed use limited to: University of Liverpool. Downloaded on November 16,2022 at 18:13:22 UTC from IEEE Xplore. Restrictions apply.

classification accuracy on each of the four minority classes than [1], which verifies again the effectiveness of our proposed network.

V. CONCLUSION AND FUTURE WORK

In this paper, we propose a new network architecture based on DSN and ResNet to classify DR into five classes. In this 11-layer network, we add three set of side-output layers at the intermediate layers and an average fusion layer on top of these side-output layers. These side-output layers provide additional regularization and learn classification features at the certain scale. Besides, to combat the class-imbalance issue of the dataset, we perform cost-sensitive learning and an oversampling method of cropping image patches. Although our network is shallower, the experiment results indicate that our method outperforms the method of [1] greatly.

Although our method shows a significant improvement, our network still encounter issues when classifying the classes of mild and moderate DR. In this case, around half of the mild and moderate images are misclassified into normal, which is a severe problem in real life. The reason of misclassification may be that the mild and moderate DR are mainly decided by the subtle parts such as microaneurysms whose feature may be lost as feed-forwarding through the network. In addition, we describe in Sect. IV that the lower level side-output layers have a better classification performance on these two classes. Therefore, it is worth to further investigate the variation of features of these images and optimize the network accordingly. At last, considering that color fundus images are usually of high-resolution and the fact that different classes of DR are highly related to different lesions, a method by combining the predictions of image level and discriminative patch level CNNs may further improve the classification performance, too.

ACKNOWLEDGMENT

This work was supported in part by the Natural Science Foundation of China under Grant 61472102, and 61672194, by the Shandong Provincial Natural Science Foundation, China under Grant ZR2016FM04, and by the Humanity and Social Science Youth foundation of Ministry of Education of China under Grant 14YJC760001.

REFERENCES

[1] Pratt H, Coenen F, Broadbent D M, et al. Convolutional Neural Networks for Diabetic Retinopathy. Procedia Computer Science, 2016, 90: 200-205.

[2] Stitt A W, Curtis T M, Chen M, et al. The progress in understanding and treatment of diabetic retinopathy. Progress in retinal and eye research, 2016, 51: 156-186.

[3] Mookiah M R K, Acharya U R, Chua C K, et al. Computer-aided diagnosis of diabetic retinopathy: A review. Computers in biology and medicine, 2013, 43(12): 2136-2155.

[4] Lee C Y, Xie S, Gallagher P, et al. Deeply-supervised nets. Artificial Intelligence and Statistics. 2015: 562-570.

[5] He K, Zhang X, Ren S, et al. Deep residual learning for image recognition. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2016: 770-778.

[6] Gardner G G, Keating D, Williamson T H, et al. Automatic detection of diabetic retinopathy using an artificial neural network: a screening tool. British journal of Ophthalmology, 1996, 80(11): 940-944.

[7] Roychowdhury S, Koozekanani D D, Parhi K K. Dream: Diabetic retinopathy analysis using machine learning. IEEE journal of biomedical and health informatics, 2014, 18(5): 1717-1728.

[8] Priya R, Aruna P. Diagnosis of diabetic retinopathy using machine learning techniques. Journal on Soft computing, 2013, 3(04).

[9] Nayak J, Bhat P S, Acharya R, et al. Automated identification of diabetic retinopathy stages using digital fundus images. Journal of medical systems, 2008, 32(2): 107-115.

[10] Acharya R, Chua C K, Ng E Y K, et al. Application of higher order spectra for the identification of diabetes retinopathy stages. Journal of Medical Systems, 2008, 32(6): 481-488.

[11] Adarsh P, Jeyakumari D. Multiclass SVM-based automated diagnosis of diabetic retinopathy. Communications and Signal Processing (ICCSP), 2013 International Conference on. IEEE, 2013: 206-210.

[12] Kotsiantis S, Kanellopoulos D, Pintelas P. Handling imbalanced datasets: A review. GESTS International Transactions on Computer Science and Engineering, 2006, 30(1): 25-36.

[13] He H, Garcia E A. Learning from imbalanced data. IEEE Transactions on knowledge and data engineering, 2009, 21(9): 1263-1284.

[14] He K, Zhang X, Ren S, et al. Identity mappings in deep residual networks. European Conference on Computer Vision. Springer International Publishing, 2016: 630-645.

[15] Shen W, Wang X, Wang Y, et al. Deepcontour: A deep convolutional feature learned by positive-sharing loss for contour detection. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2015: 3982-3991.

[16] Hwang J J, Liu T L. Pixel-wise deep learning for contour detection. arXiv preprint arXiv:1504.01989, 2015.

[17] Krizhevsky A, Sutskever I, Hinton G E. Imagenet classification with deep convolutional neural networks. Advances in neural information processing systems. 2012: 1097-1105.

[18] Tang Y, Wu X. Scene Text Detection and Segmentation based on Cascaded Convolution Neural Networks. IEEE Transactions on Image Processing, 2017.

[19] Tang Y, Wu X, Bu W. Deeply-Supervised Recurrent Convolutional Neural Network for Saliency Detection. Proceedings of the 2016 ACM on Multimedia Conference. ACM, 2016: 397-401.

[20] Xie S, Tu Z. Holistically-nested edge detection. Proceedings of the IEEE International Conference on Computer Vision. 2015: 1395-1403.

[21] Tang Y, Wu X. Saliency Detection via Combining Region-Level and Pixel-Level Predictions with CNNs. European Conference on Computer Vision. Springer International Publishing, 2016: 809-825.

[22] Szegedy C, Ioffe S, Vanhoucke V, et al. Inception-v4, inception-resnet and the impact of residual connections on learning. arXiv preprint arXiv:1602.07261, 2016.

[23] Ren S, He K, Girshick R, et al. Faster r-cnn: Towards real-time object detection with region proposal networks. Advances in neural information processing systems. 2015: 91-99.

[24] Zagoruyko S, Komodakis N. Wide residual networks. arXiv preprint arXiv:1605.07146, 2016.

[25] Xie S, Girshick R, Dollár P, et al. Aggregated residual transformations for deep neural networks. arXiv preprint arXiv:1611.05431, 2016.

[26] Jia Y, Shelhamer E, Donahue J, et al. Caffe: Convolutional architecture for fast feature embedding[C]//Proceedings of the 22nd ACM international conference on Multimedia. ACM, 2014: 675-678.

[27] Hou Q, Cheng M M, Hu X W, et al. Deeply supervised salient object detection with short connections[J]. arXiv preprint arXiv:1611.04849, 2016.

[28] http://cs231n.github.io/neural-networks-3/#hype

Authorized licensed use limited to: University of Liverpool. Downloaded on November 16,2022 at 18:13:22 UTC from IEEE Xplore. Restrictions apply.